

PROJET DE STAGE — FILIÈRE INFORMATIQUE

# Planning de projet en 5 jours

---

Portail Carrière — Application web de recrutement  
avec module de Matching IA

React (Vite) · Node.js / Express · Supabase · OpenRouter

## Périmètre réalisé

- 4 tables PostgreSQL + 1 bucket privé
- 16 endpoints REST (5 publics, 11 privés)
- 11 écrans (5 candidat, 6 espace RH)
- Module de Matching IA sur les CV

**Volume de code** : environ 5 000 lignes

Backend 1 560 | Frontend 3 500

# Vue d'ensemble

| Jour | Phase                                       | Livrable principal                                 |
|------|---------------------------------------------|----------------------------------------------------|
| J1   | Conception, base de données & configuration | Schéma opérationnel et environnements configurés   |
| J2   | Backend — API REST & sécurité               | Endpoints CRUD et authentification fonctionnels    |
| J3   | Service de Matching IA                      | Score calculé automatiquement à chaque candidature |
| J4   | Frontend — Espace Candidat & Espace RH      | Les 3 écrans RH et le parcours candidat complet    |
| J5   | Intégration, tests & documentation          | Application testée et documentée                   |

## Démarche suivie

L'ordre respecte la logique d'un projet web : la base de données conditionne le backend, qui conditionne le frontend. Le module de Matching IA a été développé **après** la validation du socle fonctionnel, afin qu'un dysfonctionnement de l'IA ne puisse jamais compromettre le dépôt d'une candidature.

Les tâches marquées **IA** correspondent à l'évolution « Matching IA », ajoutée au périmètre initial.

**Note de périmètre**

Le cahier des charges initial exclut explicitement l'intelligence artificielle (§1.2 et §1.3). Le module de Matching IA constitue donc une **évolution ajoutée en cours de projet**, et doit être présenté comme telle. De même, ce planning en 5 jours est une **synthèse rétrospective condensée** du calendrier de 4 semaines prévu en section 6 du cahier des charges.

Choix technique documenté : le modèle retenu est `openai/gpt-4o-mini` via OpenRouter, après comparaison objective avec six alternatives gratuites (voir Jour 1, tâche 1.4).

# Conception, base de données et configuration

**Objectif** : disposer d'un socle technique opérationnel — base, dépendances, variables d'environnement — avant toute écriture de logique métier.

## 1.1 — Modélisation de la base de données

- Création des 4 tables PostgreSQL sur Supabase : `Compte_RH`, `Offre`, `Candidat`, `Candidature`
- Définition des clés primaires en UUID et des clés étrangères
- Ajout de la contrainte unique (`id_offre`, `id_candidat`) empêchant la double candidature (F07)
- **IA** Ajout des colonnes `score_ia` (INTEGER nullable) et `analyse_ia` (JSONB nullable) sur la table `Candidature`
- **IA** Contrainte de validation `score_ia BETWEEN 0 AND 100` et index (`id_offre`, `score_ia` DESC) pour accélérer le tri

## 1.2 — Storage des CV

- Création du bucket `cvs` en mode **privé** — les CV sont des données personnelles (§4.5 du cahier des charges)

## 1.3 — Initialisation des projets

- Backend : installation d' `express`, `cors`, `dotenv`, `@supabase/supabase-js`, `multer`, `jsonwebtoken`, `bcryptjs`
- **IA** Installation de `pdf-parse` (extraction de texte) et `openai` (client compatible OpenRouter)
- Frontend : création du projet Vite + React, installation d' `axios` et `react-router-dom`

## 1.4 — Configuration des variables d'environnement

- `backend/.env` : `PORT`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `JWT_SECRET`, `JWT_EXPIRATION`
- **IA** Configuration de l'accès OpenRouter : `OPENAI_API_KEY`, `OPENAI_BASE_URL`, `IA_MODELE`
- `frontend/.env` : `VITE_API_URL`
- Création des fichiers `.env.example` et exclusion des `.env` via `.gitignore`

## 1.5 — Architecture du backend

- Mise en place de `server.js`, `app.js`, `config/supabaseClient.js`, `config/constantes.js`
- Script de seeding du compte RH unique ( `seedRh.js` ) — aucune interface d'inscription n'est exposée (F08bis)

## Choix du modèle d'IA — comparaison menée

---

| Modèle testé                           | Résultat                                                      |
|----------------------------------------|---------------------------------------------------------------|
| meta-llama/llama-3.1-8b-instruct:free  | Erreur 404 — n'est plus proposé gratuitement                  |
| google/gemma-4-31b-it:free             | Erreur 429 — quota fournisseur épuisé                         |
| google/gemma-4-26b-a4b-it:free         | Erreur 429                                                    |
| z-ai/glm-5.2:free                      | Erreur 429                                                    |
| minimax/minimax-m3:free                | JSON encadré de balises Markdown, non exploitable directement |
| nvidia/nemotron-3-super-120b-a12b:free | Fonctionnel — 5,8 s, JSON valide                              |
| openai/gpt-4o-mini (retenu)            | <b>Fonctionnel — 3,2 s, JSON valide, disponibilité stable</b> |

**Justification du choix :** les modèles gratuits partagent leurs quotas entre tous les utilisateurs d'OpenRouter et renvoient fréquemment des erreurs 429, ce qui rend une démonstration en direct trop risquée. Le coût de `gpt-4o-mini` reste négligeable (environ 0,0002 \$ par analyse de CV).

**Livrables J1 :** base de données opérationnelle, bucket privé créé, dépendances installées, route `GET /api/offres` renvoyant un tableau vide en JSON.

## Backend — API REST et sécurité

**Objectif** : exposer l'ensemble des endpoints métier, protégés par authentification.

### 2.1 — Authentification du Responsable RH (F08 / F09)

- `POST /api/auth/login` : vérification du mot de passe par `bcrypt.compare()` , génération d'un jeton JWT
- Middleware `auth.middleware.js` : vérification du jeton sur toutes les routes `/api/rh/*`
- Gestion de l'expiration du jeton (8 h) et message d'erreur uniforme pour éviter l'énumération de comptes

### 2.2 — CRUD des offres (F10 à F14)

- Routes publiques : liste paginée avec filtres (mot-clé, type de contrat, lieu), détail, options de filtrage
- Routes RH : création, modification, archivage réversible, suppression définitive
- Règle de gestion : masquage automatique des offres dont la date d'expiration est dépassée
- Comptage des candidatures par offre via la jointure `Candidature(count)`

### 2.3 — Dépôt de candidature (F04 à F07)

- Middleware `upload.middleware.js` : Multer en mémoire, PDF/Word uniquement, 5 Mo maximum
- Upload du CV vers le bucket privé, création ou réutilisation du candidat, création de la candidature
- Contrôle anti-doublon renvoyant un code **409 Conflict**
- Suppression du fichier uploadé si l'insertion en base échoue ensuite (rollback manuel)

### 2.4 — Gestion des candidatures côté RH (F15 à F18)

- Liste par offre, détail complet, changement de statut (En attente / Accepté / Refusé)
- Génération d'URL signées temporaires (5 minutes) pour le téléchargement des CV

### 2.5 — Gestion centralisée des erreurs

- `erreur.middleware.js` : réponses JSON uniformes et codes HTTP cohérents (200, 201, 400, 401, 404, 409, 500)

**Livrables J2** : 16 endpoints testés avec Thunder Client, routes RH renvoyant bien 401 en l'absence de jeton.

## Service de Matching IA

**Objectif :** calculer automatiquement un score de correspondance entre chaque CV déposé et l'offre concernée, afin de faire remonter les profils les plus pertinents.

### 3.1 — Extraction du texte des CV

- Développement de `extraireTextePDF(buffer)` dans `services/aiMatching.service.js`
- Utilisation de `pdf-parse` sur le buffer mémoire fourni par Multer, sans écriture sur le disque
- Nettoyage des séparateurs de page et compactage des espaces
- Gestion des cas d'échec : PDF corrompu ou CV au format Word → retour d'une chaîne vide, jamais d'exception

### 3.2 — Conception du prompt d'évaluation

Rédaction des consignes système avec une pondération à quatre critères :

| Critère d'évaluation                        | Pondération |
|---------------------------------------------|-------------|
| Compétences techniques                      | 50 %        |
| Expérience professionnelle                  | 20 %        |
| Adéquation générale du profil avec le poste | 20 %        |
| Formation et diplômes                       | 10 %        |

- Imposition d'un format de réponse JSON strict : `score` , `points_forts` , `points_faibles` , `resume`
- Garde-fous éthiques : interdiction explicite des critères discriminatoires (genre, âge, origine) et interdiction de suivre des instructions dissimulées dans un CV (*prompt injection*)

### 3.3 — Appel au modèle de langage

- Développement de `calculerScoreIA(texteCV, descriptionOffre)`
- Configuration du client : URL de base OpenRouter, `response_format: json_object` , `temperature: 0.2` pour garantir la stabilité des scores
- Détection automatique du fournisseur (OpenAI ou OpenRouter) selon le format de la clé
- **Robustesse** : extraction tolérante du JSON (nettoyage des blocs Markdown), permettant de changer de modèle sans modifier une ligne de code
- Validation systématique de la réponse : bornage du score entre 0 et 100, contrôle des types

### 3.4 — Intégration au contrôleur de candidatures

- Analyse déclenchée avant l'insertion en base, dans `creerCandidature`
- Enregistrement de `score_ia` et `analyse_ia`
- **Principe de conception** : l'échec de l'IA ne fait jamais échouer une candidature — le score vaut alors `null` et l'enregistrement se poursuit normalement
- Vérification préalable de l'existence des colonnes, pour rester compatible avec une base non migrée

### 3.5 — Tri et endpoint de relance

- Tri `.order('score_ia', ascending: false)` sur la route `GET /api/rh/offres/:id/candidatures`
- Calcul de la répartition par niveau : élevé, modéré, faible, non analysé

- Nouvel endpoint `POST /api/rh/offres/:id/analyser` permettant de scorer les candidatures enregistrées avant la mise en place du module

**Livrables J3** : score calculé automatiquement à chaque candidature. Validation par test de discrimination : **95 %** pour un profil très adapté, **10 %** pour un profil hors sujet.

## Frontend — Espace Candidat et Espace RH

**Objectif** : construire les interfaces et les connecter à l'API REST.

### 4.1 — Socle frontend

- Instance Axios unique avec intercepteurs : injection automatique du jeton, déconnexion sur 401
- Couche `services/` isolant tous les appels API des composants
- Routage React Router et composant `RouteProtegee` pour l'espace RH

### 4.2 — Espace Candidat (public)

- Page d'accueil et liste des offres avec recherche et filtres (*debounce* de 300 ms)
- Page de détail d'une offre
- Formulaire de candidature : dépôt du CV par glisser-déposer, validation côté client, écran de confirmation

### 4.3 — IA Écran 1 : Tableau de bord RH

- Trois cartes d'indicateurs : offres publiées, candidatures reçues, candidatures en attente
- Tableau des offres : statut, nombre de candidatures, date de publication
- Titre cliquable et bouton « Voir candidatures » redirigeant vers l'écran 2
- Actions par offre : modifier, archiver / republier, supprimer

### 4.4 — IA Écran 2 : Candidatures d'une offre

- En-tête : bouton « Retour au tableau de bord », titre contextuel, mention « Classées par ordre décroissant de Score IA »
- Trois cartes de répartition avec anneaux de progression dessinés en CSS pur ( `conic-gradient` ), sans librairie graphique
- Tableau trié par score : candidat, date de postulation, **badge de Score IA coloré**, statut, action
- Recherche par nom ou email, filtre par statut
- Boutons « Exporter CSV » et « Relancer analyse IA »

### 4.5 — IA Écran 3 : Fiche du candidat

- Bandeau d'identité avec encadré « Évaluation algorithmique — Score Matching IA : X % »
- Carte « Informations du candidat » : nom, email, téléphone, diplôme, école
- Carte « Analyse du profil » : points forts, points de vigilance, résumé généré
- Carte « Lettre de motivation »
- Carte « Statut de la candidature » : boutons **Accepter** (vert), **Refuser** (rouge), **Marquer en attente** (gris)
- Carte « Curriculum Vitae » : icône PDF et téléchargement par URL signée temporaire

### 4.6 — Composant réutilisable

- `ScoreIA.jsx` : badge coloré et fonction `niveauScore()`

| Score     | Niveau           | Couleur du badge |
|-----------|------------------|------------------|
| > 75 %    | Profil qualifié  | Bleu             |
| 50 – 75 % | Profil à valider | Orange           |
| < 50 %    | Profil écarté    | Rouge            |
| null      | Non analysé      | Gris             |

**Livrables J4 :** parcours candidat et parcours RH complets, interface responsive (ordinateur, tablette, mobile).

# Intégration, tests et documentation

**Objectif** : valider l'ensemble des parcours et préparer la restitution.

## 5.1 — Tests fonctionnels de bout en bout

- Parcours candidat : consultation d'une offre → candidature avec CV → confirmation
- Tests de rejet : fichier `.txt` refusé, fichier de plus de 5 Mo refusé, double candidature renvoyant 409
- Parcours RH : connexion → tableau de bord → candidatures triées → fiche → changement de statut

## 5.2 — Tests de sécurité

- Vérification qu'un appel aux routes `/api/rh/*` sans jeton renvoie **401**
- Vérification que le bucket des CV n'est pas accessible sans URL signée
- Vérification que les fichiers `.env` sont bien exclus du dépôt Git

## 5.3 — **IA** Tests de robustesse du module

- Comportement sans clé API : candidature enregistrée avec `score_ia = null`
- Comportement sur PDF illisible et sur CV au format Word
- Test de discrimination sur deux profils opposés (95 % contre 10 %)
- Comparaison de sept modèles selon trois critères : respect du format JSON, latence, disponibilité

## 5.4 — Documentation technique

- README : architecture, procédure de démarrage, documentation des 16 endpoints
- Tableau de couverture reliant chaque exigence F01 à F18 au fichier qui l'implémente
- Section dédiée au module Matching IA : mise en service, fonctionnement, limites connues

## 5.5 — Préparation de la soutenance

- Répétition chronométrée de la démonstration (5 à 6 minutes)
- Captures d'écran de secours en cas de problème réseau le jour J
- Préparation des schémas : architecture 3-tiers, modèle relationnel, flux d'une candidature

**Livrables J5** : application complète et testée, documentation à jour, démonstration prête.

# Bilan du projet

## Livrables produits

| Composant       | Volume                                |
|-----------------|---------------------------------------|
| Backend         | 16 fichiers — environ 1 560 lignes    |
| Frontend        | 27 fichiers — environ 3 500 lignes    |
| Base de données | 4 tables PostgreSQL et 1 bucket privé |
| API REST        | 16 endpoints (5 publics, 11 privés)   |
| Interfaces      | 5 écrans candidat, 6 écrans RH        |

## Preuve d'évolutivité de l'architecture

L'ajout du module de Matching IA (1 576 lignes) n'a nécessité **aucune modification** des fichiers de socle : `server.js` , `app.js` , `config/supabaseClient.js` et `routes/offres.routes.js` sont restés strictement identiques. Seules les couches métier et présentation ont évolué — c'est précisément le résultat attendu d'une bonne séparation des responsabilités.

## Limites assumées

- Analyse limitée aux CV au format PDF contenant du texte (un PDF scanné reste non scoré)
- Le score constitue une **aide à la décision** : le Responsable RH conserve seul la main sur le statut
- Absence de tests automatisés — les vérifications ont été menées manuellement
- Un seul compte RH, sans gestion de rôles (hors périmètre, annexe 8.2 du cahier des charges)
- Jeton stocké en `localStorage` ; un cookie `httpOnly` serait plus robuste face aux attaques XSS
- Absence de notification email lors d'un changement de statut (piste d'évolution)

Documentation complète des endpoints et de la mise en service du module IA disponible dans le fichier `README.md` à la racine du projet.